

Consider any node x in the binary tree.

All nodes that are reachable by going from x upwards must contain bigger values. These are the nodes on the path from x 's parent to the root. Let b be the number of these nodes.

All nodes that are reachable by going from x downwards must contain smaller values. These are the nodes in the subtrees rooted in x 's children. Let s be the number of these nodes.

From these two constraints it follows that the value in x must always be between $s + 1$ and $n - b$, inclusive.

We will now show that all these values are indeed possible.

1. Let's pick any value v from this range and place it into node x .
2. Place the values from $n - b + 1$ to n on the path from x 's parent to the root.
3. Use the values from s down to 1 to fill the rest of x 's subtree (from top to bottom, each row in arbitrary order).
4. Use the remaining values from largest to smallest to fill the rest of the tree (again, row by row, from top to bottom).

We can easily verify that this process always produces a valid max-heap.

This gives us directly an $O(n^2)$ solution: for each position we determine the range of valid values and increment the counter for each of them.

We can then easily improve the solution to run in $O(n)$ time by using sweeping instead. (We have a collection of n intervals – one for each node – and for each value we want to count the number of intervals that contain it.)